IE 206 : Introduction to Artificial Intelligence & Machine Learning

Decision Trees, Random Forest and Boosting

Manjesh K. Hanawal

February 20, 2026

Decision Trees

Motivation

- **Interpretable & Intuitive:** Decision trees mimic human decision-making by using a hierarchical structure of conditions, making them easy to understand and interpret.
- **Handles Different Data Types:** They can work with both categorical and numerical data, making them versatile for various machine learning tasks.
- **Minimal Data Preprocessing:** Unlike other models, decision trees do not require feature scaling or extensive preprocessing, simplifying implementation.
- **Foundation for Advanced Models:** Decision trees serve as the building block for powerful ensemble methods like Random Forest and Boosting, which improve accuracy and robustness.

Example

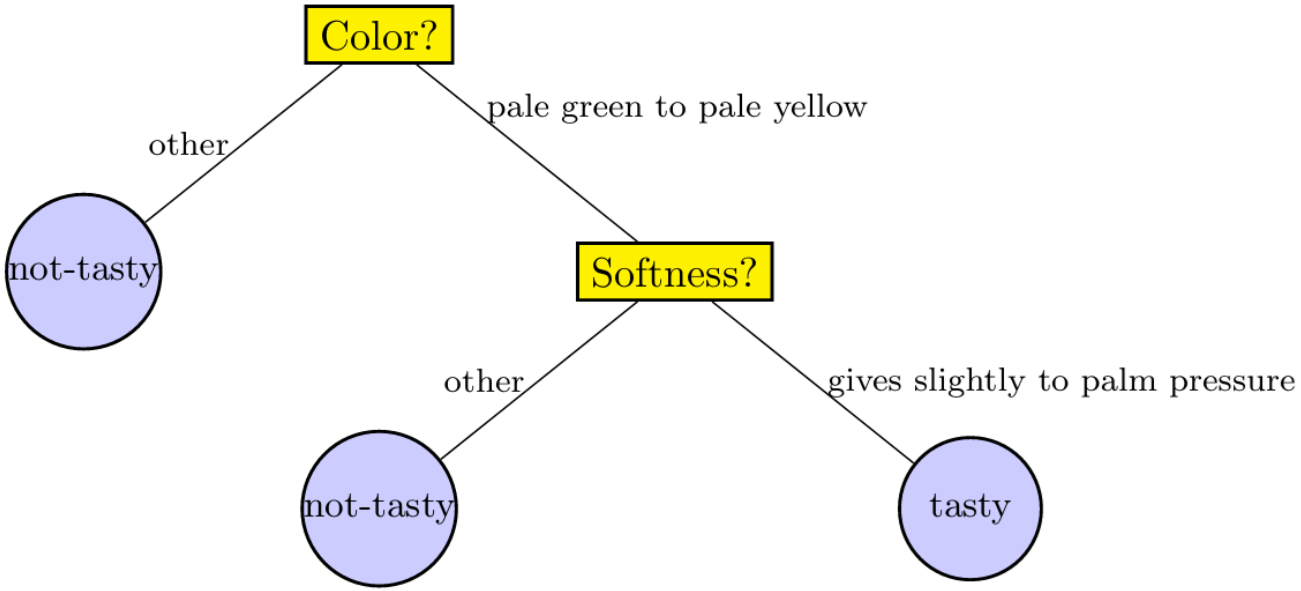


Figure: Decision tree for prediction of taste of papaya

A decision tree first checks the papaya's color. If it's outside the pale green to pale yellow range, it predicts 'not tasty.' Otherwise, it examines softness—if the papaya slightly yields to palm pressure, it's 'tasty'; otherwise, 'not tasty.' This highlights decision trees' advantage of being simple and interpretable.

Introduction

- **Definition:** A decision tree is a hierarchical model that recursively splits data based on feature conditions to classify or predict outcomes
- **Basic Structure:** Consists of nodes (decision points), branches (outcomes of decisions), and leaves (final predictions).
- **Working Principle:** Uses a divide-and-conquer approach, where data is split iteratively to minimize impurity and create homogeneous groups.

Decision Tree Algorithms

Key Idea:

- The goal is to find a decision tree that minimizes classification error.
- Finding the globally optimal decision tree is computationally hard.
- Instead, practical algorithms use a greedy heuristic approach that constructs the tree iteratively.

General Framework:

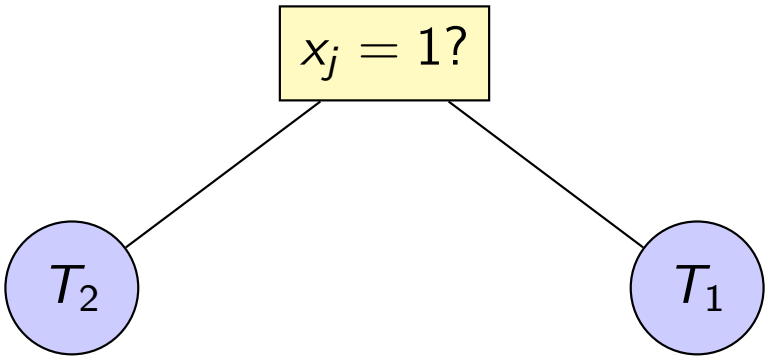
1. Start with a single leaf (root node).
2. Assign the majority class label from the training set to the root.
3. Iterate over possible splits and evaluate the gain (improvement metric).
4. Choose the best split and continue recursively.
5. Stop when no further gain is possible or a stopping criterion is met.

Algorithm

ID3 Algorithm

Input: Training set S, Feature subset $A \subseteq [d]$

1. **If** all examples in S are labeled 1, **return** a leaf labeled 1.
2. **If** all examples in S are labeled 0, **return** a leaf labeled 0.
3. **If** $A = \emptyset$, **return** a leaf labeled with the majority of labels in S.
4. **Else:**
   4.1 Let T_1 = tree returned by ID3($\{(x, y) \in S : x_j = 1\}, A \setminus \{j\}$).
   4.2 Let T_2 = tree returned by ID3($\{(x, y) \in S : x_j = 0\}, A \setminus \{j\}$).
5. **Return** the tree



Implementations of the Gain Measure

Different methods evaluate the effectiveness of a split in decision trees by using gain measures. **Information Gain**

- Measures the reduction in entropy of labels before and after a split.
- Formula for entropy:

$$C(a) = -a\log(a) - (1-a)\log(1-a)$$

- Used in ID3 and C4.5 algorithms (Quinlan, 1993).

Gini Index

- Measures impurity in data, used for classification.
- Formula:

$$C(a) = 2a(1-a)$$

- Used in CART algorithm (Breiman et al., 1984).

Extension to Real-Valued Features

- Previously, splitting rules for decision trees were defined for binary features ($\mathbb{I}[x_i = 1]$).
- For real-valued features, threshold-based splitting rules ($\mathbb{I}[x_i < \theta]$) are used.
- Threshold-based rules create *decision stumps* as splitting criteria.
- **Goal:** Reduce the problem of real-valued features to binary features.

Threshold-Based Splitting Rules

1. **Sorting and Thresholds:**
   - Sort the training instances: $x_1^i < x_2^i <, \ldots, < x_m^i$ for feature i.
   - Define thresholds $\theta_0^i, \theta_1^i, \ldots \theta_{m+1}^i$
   - $\theta_0^i = -\infty$, $\theta_0^i = \infty$, and $\theta_j^i = \frac{x_j^i + x_{j+1}^i}{2}$.
2. **Binary Feature Construction:**
   - For each i, j define a binary feature $\mathbb{I}[x_j^i < \theta_j^i]$.
   - Apply the ID3 procedure to these binary features.
3. **Efficiency Considerations:**
   - Naive Gain computation: $O(dm^2)$.
   - Optimized Gain computation: $O(dm \log(m))$.

Where d is the number of features and m is number the of training examples

Random Forest

Introduction

The Challenge with Decision Trees

- Decision trees of arbitrary size have infinite VC dimension, making them susceptible to overfitting the training data.
- Traditional solution was restricting tree size, but Random Forests offer a more powerful alternative through ensemble learning.

Random Forest Framework

- An ensemble classifier combining multiple decision trees (Breiman, 2001)
- Each tree is built using:
  - A randomly sampled training set S
  - A random vector θ for feature selection
- Final prediction combines individual tree predictions through majority voting

Random Forest: Construction Process

Building a Robust Ensemble

1. **Sample Generation:**
   - Create training set S' by sampling with replacement from original set S
   - Sample of size m drawn using uniform distribution over S
2. **Feature Selection:**
   - Generate sequence $I_1, I_2, \ldots$ of feature subsets
   - Each I_t contains k features randomly selected from $[d]$
3. **Tree Growth:**
   - Algorithm A grows a decision tree (e.g., using ID3) on sample S'
   - At each split, feature selection restricted to maximizing Gain from set I_t
   - Small k helps prevent overfitting

Boosting

Decision Stumps

A **decision stump** is a decision tree with only one level, meaning it consists of just a single decision node and two leaf nodes. It is often used in ensemble learning methods such as boosting.

- A decision stump makes a decision based on a single feature.
- The feature is chosen to minimize classification error (using a criterion like Gini impurity or Information Gain).
- It only splits the data once based on a threshold, such as $x_i < \theta$.
- While simple, decision stumps are effective as base learners in algorithms like *AdaBoost*.
- Decision stumps are fast to compute but may not provide high accuracy on their own.

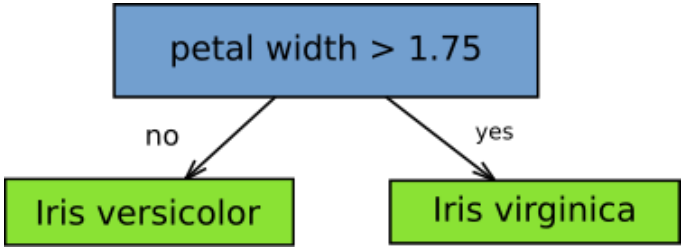


Figure: An example of a decision stump predicting flower type

The Process of Boosting

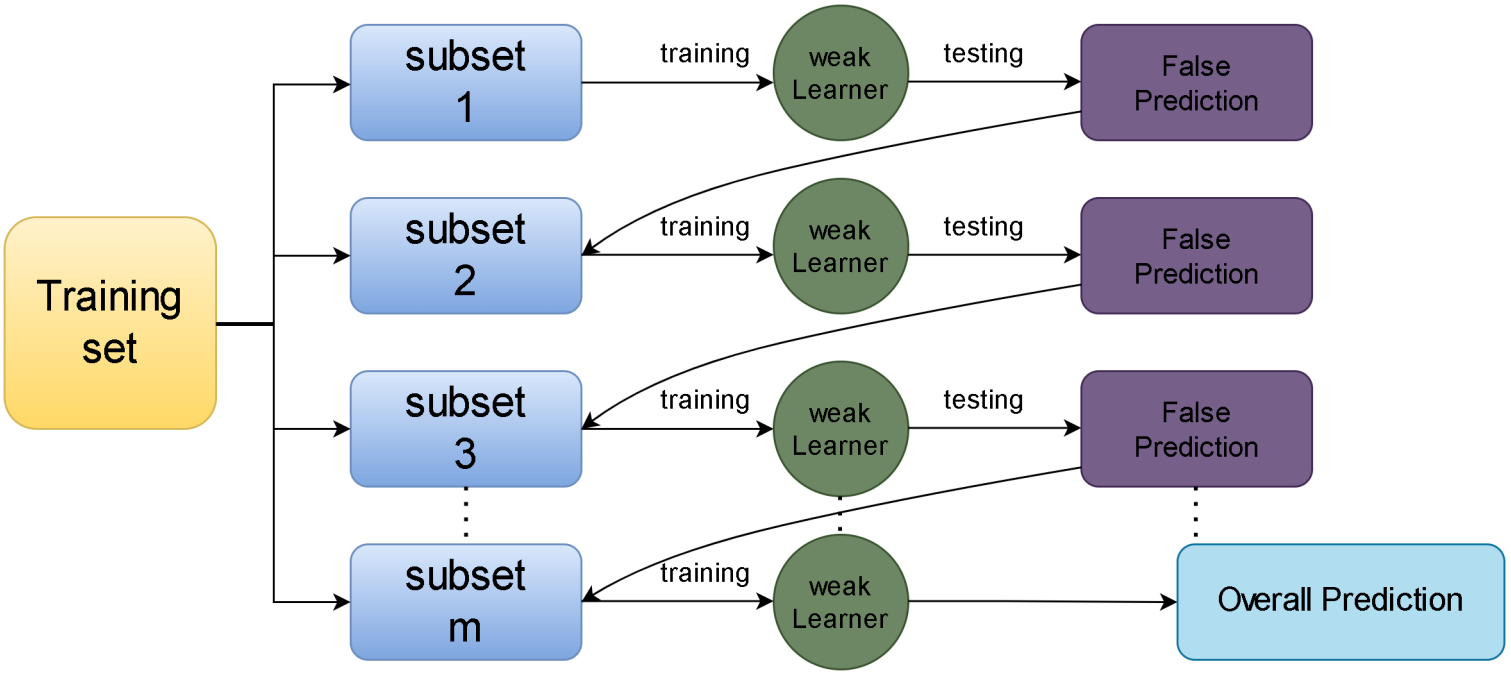


Figure: Overall boosting process

Boosting is an iterative process where weak learners are trained sequentially on subsets of data, with emphasis on misclassified samples. The predictions of all learners are then combined to form a strong overall prediction, improving the model's accuracy.

Efficient implementation of Decision Stumps using Empirical Risk minimization

The weak learner we describe later receives D and S and outputs a decision stump h : X $\rightarrow$ Y that minimizes the risk w.r.t. D.
Goal: Find (j^*, θ^*) minimizing risk:

$$L_D(h) = \sum_{i=1}^{m} D_i \mathbf{1}[h(x_i) \neq y_i]$$

Reformulation:

$$\min_{j \in [d], \theta \in \mathbb{R}} \sum_{i: y_i = 1} D_i \mathbf{1}[x_{i,j} > \theta] + \sum_{i: y_i = -1} D_i \mathbf{1}[x_{i,j} \leq \theta]$$

Optimized Approach: - Sort examples by $x_{i,j}$, define Θ_j as midpoints. - Compute $F(\theta)$ incrementally:

$$F(\theta') = F(\theta) - y_i D_i$$

- Complexity reduced from $O(dm^2)$ to $O(dm)$.

ERM for Decision Stumps

Input: Training set $S = \{(x_1, y_1), \ldots, (x_m, y_m)\}$, distribution vector D

Output: Optimal (j^*, θ^*) solving Equation (10.1)

Initialize $F^* \leftarrow \infty$ **for** $j = 1$ *to* d **do**

  Sort S using the j-th coordinate Define $x_{1,j} \leq x_{2,j} \leq \cdots \leq x_{m,j} \leq x_{m+1,j} \leftarrow x_{m,j} + 1$ Compute initial $F = \sum_{i:y_i=1} D_i$ **if** $F < F^*$ **then**

    Update $F^* \leftarrow F$, $\theta^* \leftarrow x_{1,j} - 1$, $j^* \leftarrow j$

  end

  for $i = 1$ *to* m **do**

    $F \leftarrow F - y_i D_i$ **if** $F < F^*$ *and* $x_{i,j} \neq x_{i+1,j}$ **then**

      Update $F^* \leftarrow F$, $\theta^* \leftarrow \frac{1}{2}(x_{i,j} + x_{i+1,j})$, $j^* \leftarrow j$

    end

  end

end

return (j^*, θ^*)

AdaBoost

- **AdaBoost (Adaptive Boosting):**
  - Combines multiple weak learners sequentially to form a strong classifier.
  - Minimizes empirical risk by focusing on problematic examples iteratively.
- **Input:**
  - Training set $S = \{(x_1, y_1), (x_2, y_2), \ldots, (x_m, y_m)\}$, where $y_i = f(x_i)$.
- **Boosting Process:**
  - Maintains a **distribution** $D^{(t)}$ over the examples.
  - $D^{(t)}$ assigns higher weights to examples that are hard to classify.
- **Weak Learner:**
  - Receives $D^{(t)}$ and returns a weak hypothesis h_t.
  - h_t has an error $\epsilon_t \leq \frac{1}{2}$.

Key Idea: Focus on misclassified examples by updating $D^{(t)}$ for the next iteration.

How AdaBoost Works

1. **Weight Assignment to Weak Learner:**
   - Compute the weight $w_t = \frac{1}{2} \log \frac{1-\epsilon_t}{\epsilon_t}$.
   - Weak learners with lower errors (ϵ_t) get higher weights.
2. **Updating the Distribution $D^{(t+1)}$:**
   - Increase weight for misclassified examples.
   - Decrease weight for correctly classified examples.
3. **Final Strong Classifier:**

$$H(x) = \text{sign}\left(\sum_t w_t \cdot h_t(x)\right)$$

4. **Pseudocode:**
   - Input: Training set S, weak learner, number of rounds T.
   - For $t = 1$ to T:
     - Train weak learner on $D^{(t)}$.
     - Compute w_t and update $D^{(t+1)}$.
   - Output: Weighted sum of weak hypotheses.

Theorem : Training Error Bound for AdaBoost

Theorem

Let S be a training set. Assume that at each iteration of AdaBoost, the weak learner returns a hypothesis for which:

$$\epsilon_t < \frac{1}{2}.$$

Then, the training error of the output hypothesis of AdaBoost is at most:

$$L_S(h_S) = \frac{1}{m} \sum_{i=1}^{m} \mathbb{I}[h_S(x_i) \neq y_i] \leq \exp(-2\gamma^2 T),$$

where:

- $\gamma = \frac{1}{2} - \epsilon_t$: the margin by which the weak learner outperforms random guessing.
- T: the number of boosting iterations.

Interpretation of Theorem

Key Insights:

- Theorem guarantees that AdaBoost's empirical risk decreases exponentially with more iterations.
- However, what truly matters is the generalization ability, i.e., the **true risk**.

Interpretation:

- The weak learner has error $\epsilon_t < \frac{1}{2}$ in each iteration.
- The failure probability δ can be minimized, ensuring robust learning.
- If the hypothesis space has low **VC-dimension**, the gap between empirical and true risk remains small.

Conclusion

- AdaBoost effectively reduces training error.
- With a well-chosen weak learner (e.g., decision stumps), its true risk remains low.

Discriminative vs. Generative Methods

- **Discriminative Methods:** Focus on modelling the decision boundary between the classes. They learn the conditional probability $\mathbb{P}(Y|X)$ directly from the data, aiming to find the best classification rule.
- **Generative Methods:** Attempt to model how the data was generated, aiming to understand the joint probability $\mathbb{P}(X, Y)$. They can simulate new data points and are used in both classification and generating new instances.
- **Key Difference:** Discriminative methods learn the boundaries between classes while generative methods model the distribution of individual classes.

Examples of Discriminative and Generative Methods

Discriminative Example

Support Vector Machine (SVM): SVM is a popular discriminative classifier that defines the optimal separating hyperplane maximizing the margin between different classes.

Generative Example

Naive Bayes: A simple generative model that assumes feature independence and estimates the joint probability of the features and labels to predict the class.